

PENSAMENTO COMPUTACIONAL



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

A teoria da computação e o pensamento computacional nas universidades

Luis Gustavo Maschietto

OBJETIVOS DE APRENDIZAGEM

- > Reconhecer o surgimento da ciência da computação e do pensamento computacional nas universidades.
- > Identificar o papel do pensamento computacional na programação e na automação de tarefas.
- > Descrever o impacto da computação pervasiva e do universo computacional no contexto do pensamento computacional.

Introdução

Empresas e indivíduos encontram no pensamento computacional recursos capazes de solucionar muitos de seus problemas recorrentes. Por isso, o ensino do pensamento computacional se faz cada vez mais presente e mais necessário nos diferentes níveis de estudo, sendo incorporado sobretudo em currículos de cursos universitários de diversas áreas do conhecimento.

Neste capítulo, você vai aprender como o pensamento computacional está presente no cotidiano das pessoas, como as diferentes formas de tecnologias permitem o acesso facilitado e o compartilhamento de informações e como a proximidade da tecnologia promoveu mudanças profundas em toda sociedade.

Origens da ciência da computação nas universidades

Os incentivos para uma formação acadêmica em ciência da computação iniciaram-se no mundo na década de 1950 e foram promovidos por acadêmicos, com o objetivo de aprimorar o conhecimento de novas tecnologias recém-lançadas. Graças aos departamentos de computação criados nesse período, foi possível iniciar as primeiras discussões sobre o pensamento computacional. Como as universidades da época focavam no treinamento de estudantes para atuação no mercado empresarial, não tinham o intuito de uma formação acadêmica voltada à pesquisa. Empresas buscam nessas universidades mão de obra para atuação em suas fábricas de desenvolvimento de *hardware* ou *software*. Portanto, o pensamento computacional foi fortemente influenciado para atuações empresariais (DENNING, 2019).

As universidades sempre exerceram grande influência nas decisões tomadas em relação aos tipos de formação acadêmicas ofertadas, além de gerenciarem a captação de alunos e controlarem departamentos, espaço físico e orçamento que serão destinados ao desenvolvimento de suas ações de pesquisa. Foi nesse ambiente acadêmico que surgiram os novos departamentos de ciência da computação, que tinham como prerrogativa a manutenção de uma imagem pública coesa e que representasse a identidade das universidades. No início, a criação de um departamento exclusivo para a computação foi recebida com muita resistência por parte de alguns departamentos universitários, principalmente por exigir uma divisão de turmas que antes eram aglutinadas em departamentos mais abrangentes, como o departamento de ciência e engenharia. Diversos debates foram necessários para se alcançar um consenso quanto à importância da centralização do estudo da computação para subsidiar estudos em outros setores da ciência. Nos Estados Unidos, após a aprovação da criação do primeiro departamento exclusivo para computação, o número de departamentos criados nessa área cresceu exponencialmente dos anos de 1962 até 1980. Nos dias de hoje, diversas universidades contam com departamentos de computação, e são ofertadas anualmente inúmeras vagas para o curso de ciência da computação (DENNING, 2019).

Departamentos de ciência da computação são encontrados em escolas de ciência, engenharia e gestão de negócios. Em 1990, os termos **computação** e **informática** começaram a ser utilizados e diversas universidades passaram a oferecer cursos de computação separados de outros cursos. As sociedades acadêmicas de informática como a Computer Society do Institute of Electrical and Electronics Engineers (IEEE-CS) e a Association for Computing Machinery (ACM) foram criadas para defender interesses da área da ciência da computação, como a promoção de currículos e recomendações. Esses incentivos mostram como os pioneiros da computação se esforçaram para articular a criação de uma identidade única da ciência da computação e com um pensamento computacional desvinculado de outras áreas.



Fique atento

Em termos aproximados, é possível traçar os seguintes períodos como relevantes para o sucesso dos esforços que geraram essa independência da área da computação (DENNING, 2019):

- 1950–1970 — surgimento de recursos e evolução contínua da computação;
- década de 1970 — a programação passa a ser vista como arte e ciência;
- década de 1980 — computação para automação;
- década de 1990 até hoje — computação como processos de informação pervasiva.

No Brasil, no ano de 1964 foi criado o Fundo de Desenvolvimento Tecnológico do Banco Nacional de Desenvolvimento Econômico (Funtec/BNDE), que tinha o objetivo de fomentar a criação de grupos de pesquisa e formar recursos humanos nas áreas de ciência e tecnologia. Em 1970, foi criada a Comissão de Coordenação das Atividades de Processamento Eletrônico (Capre), via decreto do Governo Federal para buscar a independência tecnológica da informática nacional mediante uma política governamental de desenvolvimento. Dessa forma, o Funtec passou a apoiar de forma mais ampla a pesquisa em tecnologia da computação. Graças a esses incentivos, diversos trabalhos importantes foram desenvolvidos, como o primeiro computador brasileiro, construído e projetado pela Escola Politécnica da Universidade de São Paulo (USP), o Processador de Aritmética de Ponto Flutuante, pela Universidade Federal do Rio de Janeiro (UFRJ), o Software do Processador PADE, pela Universidade Federal de Minas Gerais (UFMG), o Processador de Dados Estocásticos, pelo Instituto de Física da USP, entre outros (CABRAL, 2008).

Em 1972, foi criado um minicomputador pela USP em conjunto com a Companhia Equipamentos Eletrônicos. O projeto, que durou dois anos, foi chamado de G-10. A partir de 1968, reuniões foram organizadas pela comunidade acadêmica de computação, conhecidas como Seminário de Computação na Universidade (Secomu). Essas reuniões eram marcadas por ideias criativas que buscavam uma formação de recursos humanos e de política industrial de informática. Em 1974, numa das reuniões, foram estabelecidas medidas que visavam a proteção da tecnologia nacional a partir da institucionalização de incentivos especiais a empresas sob o controle nacional. Ainda em 1972, foi promovido o primeiro evento científico nacional da área de computação, chamado de Seminário sobre Desenvolvimento Integrado de Software e Hardware (SEMISH), pelo curso de Pós-Graduação em Ciência da Computação da Universidade Federal do Rio Grande do Sul. A consolidação de uma comunidade acadêmica de computação ocorreu graças a eventos como o SEMISH e o SECOMU, que permitiram um alinhamento dos interesses nacionais de desenvolvimento na área de informática. Em outros países, havia diversos esforços para reunir sociedades científicas de computação, como a International Federation for Information Processing (IFIP) e a ACM, que eram formadas por grupos de discussão sobre currículos e pesquisa e que tinha uma visibilidade internacional (CABRAL, 2008).

Em 1978, foi criada a Sociedade Brasileira de Computação (SBC) pela UFRJ. Seu objetivo era a manutenção de uma comunidade acadêmica com interesses no ensino e na pesquisa nas áreas de computação, exercício profissional e políticas industriais de informática. Com o fim da Capre, a SBC, por se tratar de uma sociedade independente e sem fins lucrativos, iniciou o estímulo à pesquisa e à formação de recursos humanos para o mercado de trabalho e a política industrial. Atualmente, a SBC possui milhares de sócios ativos, constituídos por sócios institucionais, estudantes, pesquisadores e professores. Ela promove diversos eventos anuais como simpósios, *workshops* e eventos regionais que atraem milhares de participantes, além de promover a Maratona de Programação e a Olimpíada Brasileira de Informática. É responsável por editar quatro publicações como a *Revista Eletrônica de Iniciação Científica*, o *Journal of Integrated Circuits and Systems*, a *Revista Brasileira de Informática na Educação* e o *Journal of the Brazilian Computer Society* (CABRAL, 2008).

No Brasil, os primeiros cursos na área de computação surgiram no final da década de 1960. A criação desses cursos só foi possível com a chegada dos primeiros computadores às universidades. Em 1969, a Universidade Federal da Bahia (UFBA) e a Universidade de Campinas (Unicamp) foram as primeiras universidades brasileiras a adotarem o curso de ciência da computação.

Posteriormente, na década de 1970, outras universidades como USP, UFMG, UFRJ, Universidade da Paraíba (UFPB) e Universidade de Pernambuco (UFPE) criaram cursos de bacharelado em informática. A Capre foi responsável por dar subsídios às universidades e estimular o treinamento e a pesquisa na área da computação. Diversos investimentos para a diversificação de parques de computadores nas universidades federais e estaduais foram realizados, principalmente com a instalação de computadores da marca Burroughs. Na época, marcas como IBM, Burroughs e Digital monopolizavam o mercado mundial de computadores (CABRAL, 2008).

Os primeiros cursos em ciência da computação foram criados pelas universidades de forma independente e se baseavam nos modelos norte-americanos. Apesar de capacitar os alunos com os principais conceitos da computação, não permitiam aperfeiçoamentos em novas pesquisas a partir de cursos de pós-graduação *stricto sensu*. Apenas com a nova Lei de Diretrizes e Bases da Educação Nacional, vigente no período de 1975 a 1979, é que foram estabelecidos incentivos para a criação de cursos de pós-graduação na área de computação (CABRAL, 2008).

No ano de 1996, surgiram novas modalidades de cursos no país, o que exigiu a inserção de currículos mínimos e uma maior autonomia didático-científica das universidades. Em 1999, a área de computação passou a ser chamada de computação e informática, o que permitiu a inserção de um número limitados de denominações. Dessa forma, ficaram vigentes as denominações Cursos de Licenciatura, Cursos Superiores de Tecnologia, Sistemas de Informação, Bacharelado em Engenharia da Computação e Bacharelado em Ciência da Computação. Essas denominações seguiam os padrões internacionais ditados pela ACM. Em 1985, foi criado pelo Instituto Militar de Engenharia (IME), o primeiro curso de engenharia de computação do Brasil, e em 1997, o primeiro curso de licenciatura em computação pela Universidade de Brasília (UnB).

No país, em relação às diretrizes curriculares da área de computação e informática, existem quatro categorias de cursos, detalhados a seguir (CABRAL, 2008).

- Cursos de tecnologia: têm por objetivo atender às necessidades emergenciais do mercado de trabalho. Normalmente, possuem curta duração e não possuem continuidade. Assim que a demanda por profissionais for atendida, o curso se encerra.
- Cursos de licenciatura em computação: têm por objetivo formar educadores para o ensino de computação, formar recursos humanos para o desenvolvimento de *software* educacional e formar recursos humanos para o desenvolvimento de *software* para educação a distância.

- Cursos que possuem atividades meio na área de computação: têm por objetivo formar recursos humanos para automação dos sistemas de informação das organizações. Nesses cursos, as subáreas são administração e computação. Normalmente, alunos que seguem esses cursos tendem a procurar um aperfeiçoamento por meio de especializações *lato sensu*.
- Cursos que possuem atividades fim na área de computação: cursos de bacharelado em engenharia de computação e ciência da computação, que têm como objetivo a formação de recursos humanos para o desenvolvimento tecnológico e científico. Esses cursos oferecem subsídios para que os estudantes sigam a carreira acadêmica, portanto, inseridos em atividades de pesquisa. O curso de engenharia é mais aplicado à automação industrial e envolve física e partes elétricas para a resolução de problemas.

Em 2006, com o objetivo de padronizar a oferta desses cursos, foi criado o Catálogo Nacional de Cursos Superiores de Tecnologia, permitindo que os estudantes dessem continuidade aos estudos na área de computação via cursos de pós-graduação.



Saiba mais

Lançado em 2006, o Catálogo Nacional dos Cursos Superiores de Tecnologia é um guia de informações sobre o perfil de competências do tecnólogo. Apresenta a carga horária mínima e a infraestrutura recomendada para cada curso (BRASIL, 2016).

O ensino de habilidades de computação vem sendo estimulado em diversas partes do mundo e em diversos níveis de escolaridade. Porém, ainda é inconclusivo se a qualidade desse ensino é suficiente para que tais conhecimentos sejam aplicados em diferentes domínios. O conhecimento computacional, por ser fundamental na resolução de vários problemas, pode estar presente no currículo escolar de diversos cursos existentes. No Reino Unido, por exemplo, o ensino computacional está presente na grade escolar e cumpre um papel importante na resolução de problemas que exigem testes computacionais. O suporte computacional é utilizado na construção de modelos, desenho de diagramas, análise de dados científicos, entre outros recursos necessários em pesquisas científicas. Nesse aspecto, a ideia não é ensinar todas as pessoas a pensarem como cientistas da computação, mas ensiná-las a aplicar elementos comuns para resolver problemas e descobrir novas questões que podem ser exploradas dentro de todas as disciplinas (BEECHER, 2017).

Desse modo, ensinar o pensamento computacional não é o mesmo que ensinar ciência da computação. Na ciência da computação, o objetivo é educar o aluno em aplicações e princípios da computação matemática. O ensino de programação, subárea do estudo da ciência da computação, é aplicado principalmente com o objetivo de ensinar aos alunos as melhores práticas para desenvolver programas, concentrando-se na produção de *software* de alta qualidade. Já o pensamento computacional é definido como uma abordagem para a solução de problemas via computador (BEECHER, 2017).

O pensamento computacional na programação e automação de tarefas

Diversos desenvolvimentos relevantes surgiram na área da computação na década de 1960, graças aos esforços e à dedicação em novas pesquisas tecnológicas realizadas pelos cientistas da computação. Uma das tecnologias que trouxeram benefícios sociais e que promoveram um pensamento computacional sistêmico foram os sistemas operacionais, com esforços voltados a novas aplicações acessíveis e interativas aos usuários. Preocupações relacionadas à engenharia da computação na produção de sistemas confiáveis surgiram no final dos anos 1960 e favoreceram um pensamento computacional voltado à engenharia. Em 1967, com o início do projeto Arpanet, o pensamento computacional direcionou seus ideais para estruturas de rede. A partir da consolidação das tecnologias na área da computação, as atenções foram voltadas ao desenvolvimento da programação, o que permitiu que diversas linguagens surgissem. Diversas rotinas foram agregadas à produção de sistemas para evitar problemas relacionados à construção de produtos mal estruturados. Além disso, surgiram métodos eficientes para mensurar o custo computacional empregado na execução de um algoritmo (DENNING, 2019).



Saiba mais

A automação de trabalhos só foi possível graças às linguagens de programação, que facilitaram a avaliação de modelos matemáticos e deduções lógicas. Em 1957, a linguagem Fortran (Formula Translation) era capaz de avaliar esses modelos, e as deduções foram possíveis em 1958, com o uso da linguagem LISP (List Processing). Em 1959, a linguagem Cobol (Common Business Oriented Language) era capaz de rastrear inventários de negócios e manter banco de dados de clientes. Em 1958, com a criação da linguagem Algol (Algorithmic Language) era possível comunicar especificações precisas de algoritmos para serem incorporadas com outras linguagens (DENNING, 2019).

Basicamente, programas de computador podem ser definidos como expressões de algoritmos em uma linguagem formal, que posteriormente são compilados em linguagem de máquina para serem executados. A programação está presente em diversas áreas da computação e serve de auxílio para novas pesquisas desenvolvidas em ciência da computação. A linguagem Assembly era encontrada nos primeiros computadores na década de 1940, composta de códigos com instruções linha a linha para o controle das execuções de máquina. No Assembly, por exemplo, uma instrução como “ADD R1, R2, R3” permitia que fossem somados os registros R1 e R2 no registro R3. Apesar de eficiente, a linguagem Assembly era complexa demais. Como forma de melhorar o entendimento na digitação de comandos de baixo nível (nível de máquina), foram desenvolvidas linguagens capazes de permitir um nível superior de entendimento na construção de comandos (semelhante à estrutura de escrita humana) a partir de códigos preestabelecidos, o que permitiu uma simplificação no trabalho de um programador e uma diminuição de erros de digitação (DENNING, 2019).

Os paradigmas de programação passaram a indicar linguagens que atendem formas particulares de pensar sobre um problema. Nesse sentido, as linguagens Fortran, Cobol e Algol são classificadas como linguagens que apresentam o mesmo paradigma, pois veem os programas como uma série de módulos ou procedimentos que utilizados para comandar uma máquina. Ainda existem paradigmas definidos como orientados a objetos (programação focada na interação de objetos que trocam mensagens entre si e com o mundo externo), assim como as linguagens Smalltalk e Java. Na década de 1970, o paradigma de programação era visto como um estilo diferente do pensamento em algoritmo. Em 1980, o pensamento computacional era conhecido como um conjunto de conhecimentos e habilidades relacionados aos algoritmos e ao desenvolvimento de sistemas (DENNING, 2019).

Outro recurso indispensável para o desenvolvimento de *software* é o raciocínio lógico. Nesse caso, a lógica está presente desde o levantamento dos requisitos funcionais até a implantação do sistema. Um *software* de computador depende de instruções lógicas para que seja possível sua execução a partir de um caminho escolhido. Programas *web*, por exemplo, precisam saber o momento certo de executar a venda de um produto escolhido, enquanto um aplicativo de câmera digital deve estabelecer o tempo necessário para conseguir capturar uma foto. Cada uma dessas escolhas é realizada a partir de instruções de computador determinadas pela lógica. Diversas linguagens de computador utilizam a instruções condicional *if* (se) para as tarefas do sistema. Sua principal função é forçar o programador a pensar de maneira lógica em relações de causa e efeito. Desse modo, uma condição lógica (a

causa) força o programa a realizar alguma tarefa (o efeito). A maior parte do trabalho de programação é dedicada à identificação desses tipos de relações de causa e efeito e à tradução dessas ações em instruções (RILEY, 2014).

É importante destacar que o pensamento em algoritmo difere do pensamento matemático em vários aspectos, tais como (DENNING, 2019):

- formulação de representações da realidade;
- atenção voltada às ações que alteram os estados dos dados;
- ênfase em estruturas de informação;
- redução de problemas complexos em simples.

Atualmente, as ferramentas mentais do pensamento computacional são menos matemáticas que as primeiras descrições de pensamento em algoritmo. Especialistas na área de computação defendem que a forma de pensar da computação é uma ferramenta genérica no nível da matemática e da linguagem. Outras opiniões são divergentes e consideram que o pensamento em algoritmo é uma habilidade rara e inata. A ideia de que esse pensamento é genérico tem mais apoio dos educadores, porque estimula a ideia de que todos podem aprender o pensamento computacional. Assim, esse modo de pensar seria algo a ser aprendido, e não uma capacidade com a qual nascemos (DENNING, 2019).

No final dos anos 1970, havia na área da computação diversas dúvidas a serem respondidas em relação à produtividade de desenvolvimento de sistemas, criação de novas linguagens, segurança de sistemas, sistemas em rede, entre outras. Essas dúvidas indicavam que o pensamento computacional estava presente em diversos recursos para a automação de tarefas. Automatizar algo indica a existência de processos com o objetivo de controlar dispositivos com o mínimo de intervenção humana (DENNING, 2019).

Na década de 1970, foi desenvolvida a teoria da complexidade computacional, que tinha por objetivo classificar os problemas de acordo com etapas necessárias em algoritmos para resolvê-los. Para exemplificar, um computador que é capaz de realizar 1 bilhão de instruções por segundo consegue finalizar a pesquisa de uma lista de 100 itens em 1 microssegundo. Já para imprimir e numerar todos os subconjuntos de 100 itens, seriam necessários cerca de 10^{14} anos. Isso é 10 mil vezes mais que a idade do universo (DENNING, 2019).

Algoritmos heurísticos são soluções eficientes para resolver problemas complexos. Eles avaliam cada item a ser resolvido considerando seu peso, valor e proporção e, em seguida, empacotam em ordem decrescente de proporções até chegar ao limite de sua capacidade. Para diversas situações, as soluções

presentes nos algoritmos heurísticos apresentam grande eficiência, mas existem recursos capazes de melhorar esse trabalho. Atualmente, há uma compreensão sofisticada da complexidade computacional, da lógica e dos métodos de otimização que podem projetar bons algoritmos heurísticos. A inteligência artificial (IA) apresenta diversos recursos que exploram o conceito de automação por computador. Entre esses recursos estão a reprodução de tarefas cognitivas e de movimentos humanos (DENNING, 2019).

Sistemas gerenciadores de banco de dados são projetados para automatizar o armazenamento de uma grande quantidade de dados de forma eficiente e relacional. Suas rotinas permitem que uma informação seja localizada e recuperada em pouco de tempo. Tais ferramentas são componentes essenciais em quase todos os sistemas que permitem realizar uma automação e são empregados em diversas áreas. A maioria dos bancos de dados requer *software* sofisticado de extração e análise de dados de interesse. Apesar disso, os bancos de dados são projetados para localizar registros de forma eficiente em respostas e consultas bem estruturadas. A Structured Query Language (SQL) é uma linguagem que permite a realização de consultas em uma base de dados (RILEY, 2014).

Ao produzir um sistema para um cliente, o engenheiro de *software* é responsável por definir de forma precisa o que o *software* deverá fazer. Essa definição segue princípios da engenharia de requisitos, que prevê diversas reuniões com o cliente para estabelecer as funcionalidades necessárias. Portanto, o engenheiro de *software* é responsável por garantir que todos os detalhes foram abordados na análise de sistemas para o entendimento comum dos aspectos do *software*. Ao final do processo de levantamento de requisitos, é gerado um documento conhecido como documento de requisitos de *software*. As proposições lógicas são frequentemente usadas para definir funções importantes de um grande projeto de *software* (RILEY, 2014).

Após a análise das principais tecnologias presentes para construir um programa computacional, fica claro que é impossível entender o pensamento computacional sem compreender as habilidades de resolução de problemas e técnicas da ciência da computação. Existem quatro estratégias empregadas por cientistas da computação para solucionar um problema: definição do problema, raciocínio lógico, decomposição e abstração. Assim, o processo de resolução sempre começa pela definição do problema. Nessa etapa, são definidas as tarefas que devem ser realizadas pelo programa, ou seja, o objetivo do sistema. Os engenheiros são responsáveis por cumprir ao menos três fases do processo de desenvolvimento: análise, *design* e implementação. Dentre as três fases, a análise é considerada a mais importante, pois nela são

estabelecidas as regras do sistema. Esse processo deve envolver pessoas que conhecem o escopo do sistema. Os requisitos a serem identificados para o início do desenvolvimento do sistema são divididos em requisitos funcionais (tarefas específicas do sistema) e requisitos não funcionais (outras características e restrições do sistema) (RILEY, 2014).

Computação pervasiva e o universo computacional

No final dos anos 1990, a fabricante Internacional Business Machines (IBM) criou o termo computação pervasiva, que se refere ao processo técnico de inserção de *microchips* nos mais variados objetos. Esse tipo de tecnologia abre espaço para uma grande disponibilidade de computadores que executam serviços e funções em diversos ambientes. Dessa forma, os computadores deixaram de ser máquinas fixas em ambientes predeterminados, como os computadores *desktop*, e passaram a estar presentes em diversos objetos e lugares distintos (ANDRADE, 2015).

Atualmente, diversos recursos computacionais estão presentes no dia a dia das pessoas. Portanto, a computação passou a resolver problemas de forma inteligente e onipresente, a partir de dispositivos com tecnologias embutidas. Todos esses dispositivos criam uma computação ubíqua, por estarem agregados às estruturas básicas e fundamentais do nosso cotidiano (MORAES, 2020).

A informação disponibilizada por esses dispositivos onipresentes é compartilhada por outros dispositivos e os aplicativos acompanham seus usuários nas tarefas do dia a dia. Essas tarefas podem envolver a busca por uma localização de um ponto específico, uma postagem em uma rede social, a captura de uma imagem de interesse a partir da câmera do celular ou simplesmente a comunicação por voz entre usuários. A interação entre dispositivos pode ocorrer por tecnologias como USB, Bluetooth, infravermelho ou via rede Wi-Fi. Estudos relacionados à inteligência ambiental visam o desenvolvimento de espaços inteligentes para adequação de interesses, desejos e necessidades das pessoas. Isso evidencia a presença da ciência da computação na sociedade da informação (MORAES, 2020). A Figura 1 ilustra a comunicação de um usuário em uma rede de determinada organização. A partir do acesso à rede corporativa, o usuário é capaz de acessar a intranet dessa organização, sua intranet doméstica ou a internet (COULOURIS, 2013).

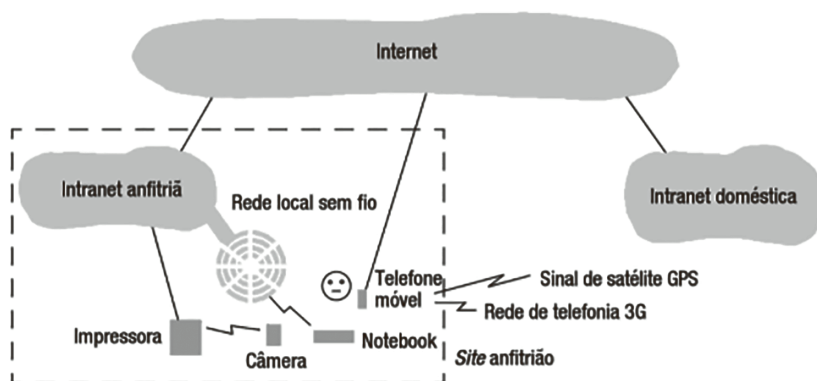


Figura 1. Equipamentos portáteis em um rede distribuída.

Fonte: Coulouris (2013, p. 11).

Na situação mostrada na Figura 1, o usuário tem acesso a três tipos diferentes de conexão sem fio. A partir do seu computador, é capaz de se conectar à rede local sem fio anfitriã de uma organização. Essa rede permite uma conexão de longa distância capaz de ser acessada de outros pontos de um mesmo ambiente e com acesso livre à intranet dessa organização a partir de um *gateway* (ponto de acesso). Com o telefone do usuário, é possível realizar a conexão com a internet e ter acesso a diversos tipos de serviços disponibilizados via *web*. O telefone do usuário também permite a visualização de informações locais via GPS que estão incorporadas em seu dispositivo. Além de todo o acesso à rede, ele ainda será capaz de executar tarefas básicas do seu dispositivo, como tirar fotos ou enviar dados para outro dispositivo, como uma impressora (COULOURIS, 2013).

A internet móvel permite não apenas a proliferação de informações, mas também sua diversificação. O pensamento computacional permitiu uma transformação do universo da imagem e da linguagem hipermídia, que promoveu um maior entendimento e um rápido compartilhamento das informações. Essa hipermobilidade (mobilidade física com recursos móveis) permite a inserção e o acesso de informações de diversos tipos, sejam elas de geolocalização, práticas, *ad hoc*, conceituais etc. Redes sem fio se tornaram essenciais para esse tipo de comunicação ubíqua, corporificada e multiplamente situada em objetos do cotidiano que compõem as tecnologias embarcadas e a internet das coisas (LIMA, 2013).

Diversos estudos defendem que existe outro estágio de evolução em que o próprio universo seria um computador. Dessa forma, o pensamento computacional não seria apenas uma habilidade a ser aprendida, mas o

comportamento natural do cérebro. Sob esse prisma, o conceito de IA forte refere-se à crença de que máquinas devidamente programadas poderiam ser, de fato, inteligentes. Já a IA fraca defende que, por meio de uma programação inteligente, as máquinas seriam capazes de simular atividades mentais para parecerem inteligentes. As assistentes virtuais Alexa e Siri, por exemplo, são consideradas IA fracas, pois realizam o reconhecimento de comandos comuns, mas não são capazes de compreendê-los. Semelhante à distinção forte/fraco em IA, a visão “forte” do universo computacional afirma que o próprio universo, junto com todos os seres vivos, é um computador digital. Nessa teoria, cada dimensão de espaço e tempo é discreta e todo movimento de matéria ou energia é um cálculo. Já a versão “fraca” não chega a afirmar que o mundo é um computador, mas que as interpretações computacionais do mundo são úteis para estudar fenômenos com a possibilidade de realizar modelagens, simulações e estudos usando um computador. Porém, é importante ressaltar que a visão computacional forte é altamente especulativa e enfrenta inúmeros problemas empíricos e filosóficos (DENNING, 2019).

Referências

- ANDRADE, L. A. *Jogos digitais, cidade e (trans) mídia: a próxima fase*. Curitiba: Appris, 2015.
- BEECHER, K. *Computational thinking: a beginner's guide to problem-solving and programming*. Swindon, UK: British Computer Society, 2017.
- BRASIL. Ministério da Educação. *Catálogo Nacional dos Cursos Superiores de Tecnologia*. Brasília, DF: MEC, 2016. Disponível em: <http://portal.mec.gov.br/catalogo-nacional-dos-cursos-superiores-de-tecnologia->. Acesso em: 29 dez. 2020.
- CABRAL, M. I. C. et al. *A Trajetória dos cursos de graduação da área de computação e informática: 1969–2006*. Rio de Janeiro: SBC, 2008.
- COULOURIS, G. et al. *Sistemas distribuídos: Conceitos e Projeto*. 5. ed. Porto Alegre: Bookman, 2013.
- DENNING, P. J.; TEDRE, M. *Computational thinking*. Cambridge, MA: MIT Press, 2019.
- LIMA, J. C. D. *Uma abordagem de recomendação sensível ao contexto para apoio a autenticação implícita em ambientes móveis e pervasivos baseado em conhecimento comportamental do usuário*. 2013. 161 f. Tese (Doutorado em Engenharia e Gestão do Conhecimento) — Programa de Pós-Graduação em Engenharia e Gestão do Conhecimento, Universidade Federal de Santa Catarina, Florianópolis, 2013. Disponível em: <https://repositorio.ufsc.br/bitstream/handle/123456789/107473/320893.pdf?sequence=1&isAllowed=y>. Acesso em: 29 dez. 2020.
- MORAES, R. B. (org.). *Indústria 4.0: impactos sociais e profissionais*. São Paulo: Blucher, 2020.
- RILEY, D. D.; HUNT, K. A. *Computational thinking for the modern problem solver*. Flórida: CRC press, 2014.

Leituras recomendadas

ASCENCIO, A. F. G.; CAMPOS, E. A. V. *Fundamentos da programação de computadores: algoritmos, Pascal, C/C++ (Padrão ANSI) e Java*. 3. ed. São Paulo: Pearson, 2012.

BALTZAN, P.; PHILLIPS, A. *Sistemas de informação*. Porto Alegre: AMGH, 2012.

BROOKSHEAR, J. G. *Ciência da computação: uma visão abrangente*. 11. ed. Porto Alegre: Bookman, 2013.

CABRAL, C.; CAPRINO, W. (org.). *Trilhas em segurança da informação: caminhos e ideias para a proteção de dados*. Rio de Janeiro: Brasport, 2015.



Fique atento

Os *links* para *sites da web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integridade das informações referidas em tais *links*.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS